

Efficient Neural PDE-Solvers using Quantization Aware Training

Winfried van den Dool¹, Tijmen Blankevoort², Max Welling¹, Yuki M. Asano¹

¹QUVA Lab at the [University of Amsterdam](https://www.uva.nl) ²Qualcomm AI Research*

w.v.s.o.vandendool@uva.nl

Abstract

In the past years, the application of neural networks as an alternative to classical numerical methods to solve Partial Differential Equations has emerged as a potential paradigm shift in this century-old mathematical field. However, in terms of practical applicability, computational cost remains a substantial bottleneck. Classical approaches try to mitigate this challenge by limiting the spatial resolution on which the PDEs are defined. For neural PDE solvers, we can do better: Here, we investigate the potential of state-of-the-art quantization methods on reducing computational costs. We show that quantizing the network weights and activations can successfully lower the computational cost of inference while maintaining performance. Our results on four standard PDE datasets and three network architectures show that quantization-aware training works across settings and three orders of FLOPs magnitudes. Finally, we empirically demonstrate that Pareto-optimality of computational cost vs performance is almost always achieved only by incorporating quantization.

1. Introduction

In many scientific fields, mathematical models of observed phenomena are expressed in terms of Partial Differential Equations, with the solution commonly represented by a function of a single time variable and one or more spatial variables. While many PDEs can be described compactly (e.g. the famous Schrödinger’s equation from quantum physics, or Navier-Stokes’ equations from fluid dynamics) it is usually impossible to write down explicitly the formulas for their exact solutions. Rather, there is a vast amount of scientific research on methods to numerically approximate solutions.

When it comes to practical applications, computational cost and available resources play a significant role in this field of research. For most methods a natural trade-off occurs where one may use available resources to either speed

up the computation, or increase the resolution on which the PDE is defined, leading to a slower but more accurate result. This latter strategy is of particular interest for PDEs that are highly non-linear and potentially exhibit chaotic behavior. Small eddies in a turbulent fluid may, for example, be overlooked when the spatial resolution is too low, leading to increasing errors in the solution function over time. One of the most important fields where computational resources form a bottleneck is climate prediction. The challenge of determining how many degrees the planet will warm over the next decades is a computational one, with supercomputers running massive ensemble-type methods. Insufficient resolution is quite often the main cause for sub-optimal modeling accuracy [21, 1, 16]. Indeed, over time climate models have become better for a large part simply because the resolution of the grid that they are defined on was allowed to increase, due to more efficient computation and more available resources [24, 26, 11].

It is precisely in this context of the computational cost of the traditional solvers that neural PDE solvers are becoming an interesting alternative. A single forward pass in a neural network is extremely fast compared to the iterative solving procedure of classical methods, and most of the operations are large matrix multiplications that can be easily handled by GPUs. While a neural network does need training first, possibly requiring data generated by classical solvers, its efficiency benefit comes from “recycling”: After training is complete, the neural network has learned to generalize to different initial conditions, whereas conventional solvers only solve the PDE for one specific configuration of initial conditions at a time. In practice, a deep learning weather model would have to be trained only once, but could then be applied every day to predict the next day’s weather. Furthermore, when training data is sufficiently abundant in the real world, using that data directly comes with the added advantage that complex phenomena do not need to be first accurately modeled. Indeed, neural PDE solvers trained on real weather data alone have shown promising results [3, 7, 16].

Our goal is not to improve on state-of-the-art models in the conventional sense. Rather, our starting point is to borrow from existing networks and architectures and focus on

*an initiative of Qualcomm Technologies, Inc.